EGCO 425 – Chapter 7

Evaluation of Classification Models

References

[1] Tan et al., Introduction to Data Mining (Chapters 4-5)

[2] Han & Kamber, Data Mining: Concepts and Techniques (Chapter 8)

[3] Kotu & Deshpande, Data Science: Concepts and Practice (Chapters 8, 15)

[4] RapidMiner Doc, https://docs.rapidminer.com/latest/studio/operators/

R. Marukatat

Model Evaluation Roadmap

- Run classifier on testing data to get preliminary performance
  - Overall performance → accuracy
  - Per-class performance → precision, recall, F-measure
  - Overfitting/underfitting → training vs. testing performance
  - Adjust parameters & preprocessing, or try other classifiers to obtain good performance → optimization
- Formal performance statistics
  - Run classifier in validation mode to get formal statistics
  - Create final working model from full data set
- Other evaluation
  - Sensitivity, specificity, J-index
  - ROC curve (TP rate vs. FP rate)

Overall Performance

- Confusion matrix (when running a classifier on a data set)

	Actual A	Actual B	Actual C	
Predicted A	f_{aa}	f_{ab}	f_{ac}	All pred A
Predicted B	f_{ba}	f_{bb}	f_{bc}	All pred B
Predicted C	f_{ca}	f_{cb}	f_{cc}	All pred C
	All actual A	All actual B	All actual C	total

f_{pa} = #records predicted as p & actual class is a

- Overall accuracy and error

$$accuracy = \frac{f_{aa} + f_{bb} + f_{cc}}{total} \quad ; \quad error = 1 - accuracy$$

- Limitation of accuracy
  - Suppose that records in class 0 = 9900; in class 1 = 100
  - Every record is predicted to be in class 0
    - Accuracy = 99 %
    - But not a good model because it is unable to detect class 1
- Solution : penalty or cost analysis

Penalty matrix		Actual	
		+	-
Predict	+	-1	10
	-	100	0

Penalty is specified by users, e.g.

- Failing to detect (+) is a severe mistake → high penalty
- Falsely detect (+) is not a severe mistake → low penalty

Penalty matrix		Actual	
		+	-
Predict	+	-1	10
	-	100	0

Can use profit instead of penalty & consider classifier with higher profit

Classifier (1)		Actual	
		+	-
Predict	+	150	60
	-	40	250

Accuracy = (150+250) / 500
= 0.80

Penalty = -150 + 600 + 4000
= 4450

Classifier (2)		Actual	
		+	-
Predict	+	245	5
	-	50	200

Accuracy = (245+200) / 500
= 0.89

Penalty = -245 + 50 + 5000
= 4805

Slightly lower accuracy, but also lower penalty

Per-class Performance

- A classifier with high accuracy may be good at predicting one class but bad at predicting another
- To analyze per-class performance
  - Positive = class in focus e.g. class A
  - Negative = other classes e.g. classes B + C
  - Divide confusion matrix into 4 areas w.r.t. positive class

	Actual A	Actual B	Actual C	
Predicted A	TP(A)	FP(A) *False alarm for A*		All pred A
Predicted B	FN(A) *Missing A*	TN(A)		All pred B
Predicted C				All pred C
	All actual A	All actual B	All actual C	total

TP = true positive
TN = true negative
FP = false positive
FN = false negative

	Actual A	Actual B	Actual C	
Predicted A	TP(A)	FP(A)		All pred A
Predicted B	FN(A)	TN(A)		All pred B
Predicted C				All pred C
	All actual A	All actual B	All actual C	total

- **Recall (A)** or TP rate (A) = ability to retrieve actual A from the data set
  $= TP(A) / \text{all actual}(A) = TP(A) / (TP(A) + FN(A))$
  $FN(A)$ = penalty of missing target records

- **Precision (A)** = reliability of positive prediction, i.e. how precise it is
  $= TP(A) / \text{all pred}(A) = TP(A) / (TP(A) + FP(A))$
  $FP(A)$ = penalty of retrieving irrelevant records

	Actual A	Actual B	Actual C	
Predicted A	TP(A)	FP(A)		All pred A
Predicted B	FN(A)	TN(A)		All pred B
Predicted C				All pred C
	All actual A	All actual B	All actual C	total

- Dilemma
  - $\text{Recall (A)} = \text{TP(A)} / (\text{TP(A)} + \text{FN(A)})$

    To get perfect recall → minimize FN(A) by making the classifier bias toward class A → may lead to higher FP(A) & worsen precision
  - $\text{Precision (A)} = \text{TP(A)} / (\text{TP(A)} + \text{FP(A)})$

    To get perfect precision → minimize FP(A) by making the classifier bias against class A → may lead to higher FN(A) & worsen recall
  - When one is high & the other is low ➔ is a classifier good or bad at predicting this class ?

- **F-measure** or F1-score (A) = trade-off between precision and recall

$$F - measure(A) = \frac{2 \times precision(A) \times recall(A)}{precision(A) + recall(A)}$$

Excel function = HARMEAN(precision, recall)

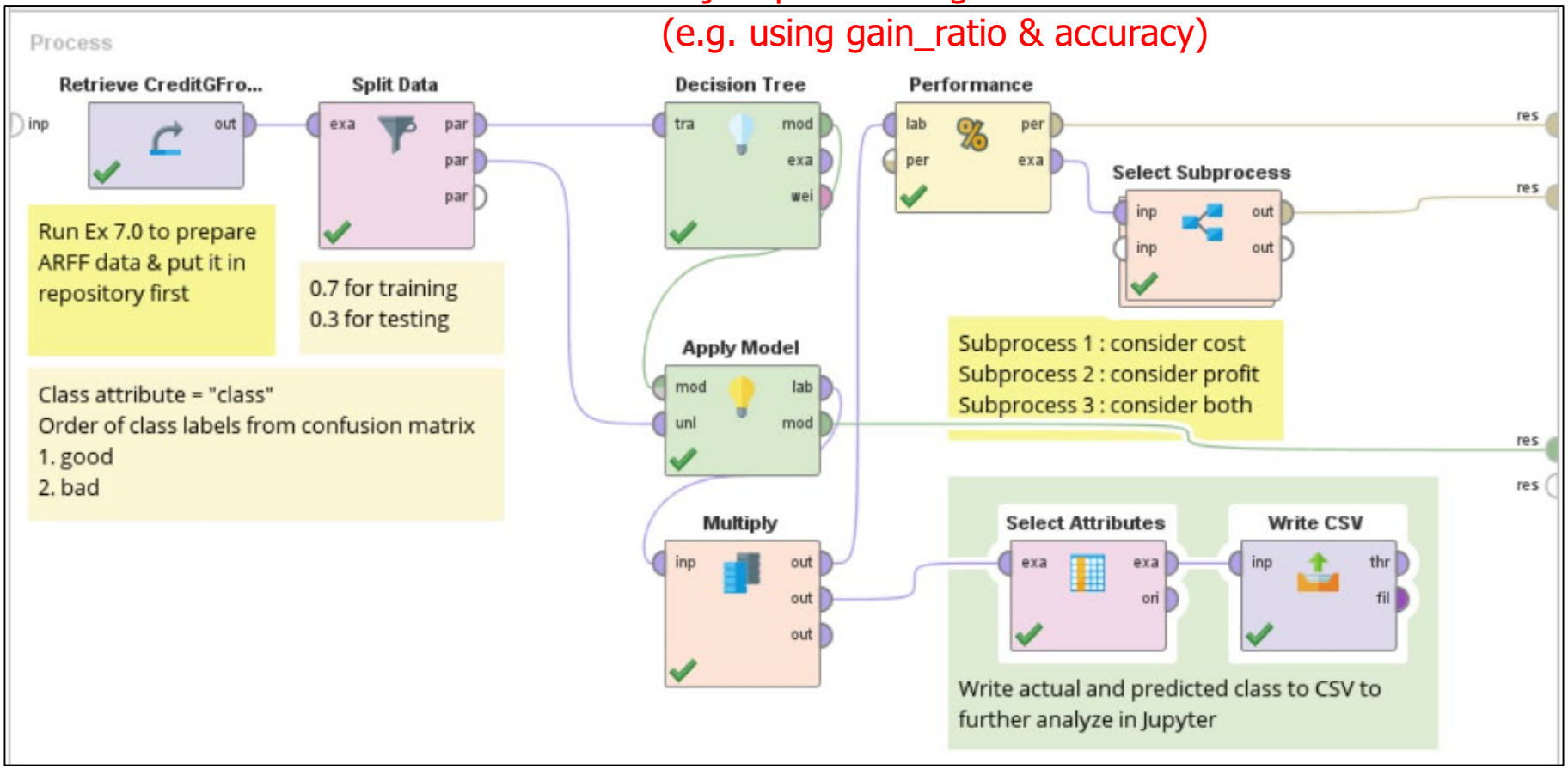
Precision	Recall	Arithmetic	Geometric	Harmonic
1	5	3	2.24	1.17
8	12	10	9.80	9.60

$$arith\ mean = \frac{precision + recall}{2} \quad ; \quad geo\ mean = \sqrt{precision \times recall}$$

Harmonic mean is pulled toward smaller value more than other means
Big harmonic mean ensures that both precision & recall are high

Example (1) : comparing classifiers

Adjust params to get 2 classifiers
(e.g. using gain_ratio & accuracy)



Total testing records = 300

accuracy: 70.33%

Classifier 1 (gain_ratio)

	true good total = 210	true bad total = 90	class precision
pred. good total = 199	160	39	80.40%
pred. bad total = 101	50	51	50.50%
class recall	76.19%	56.67%	

accuracy: 73.33%

Classifier 2 (accuracy)

	true good	true bad	class precision
pred. good total = 246	188	58	76.42%
pred. bad total = 54	22	32	59.26%
class recall	89.52%	35.56%	

Classifier 2 has higher overall accuracy

Prediction from classifier 1 is quite balance (199 good & 101 bad predictions)
Prediction from classifier 2 is heavily biased toward good (246) & against bad (54)

Report by Python

```
ResultDF1 = pd.read_csv('./data/output7-1-classifier1.csv', sep = ';')
ResultDF1.head()
```

	id	class	confidence(good)	confidence(bad)	prediction(class)
0	3	good	0.925581	0.074419	good
1	5	bad	0.490683	0.509317	bad
2	6	good	0.925581	0.074419	good
3	11	bad	0.646707	0.353293	good
4	13	good	0.646707	0.353293	good

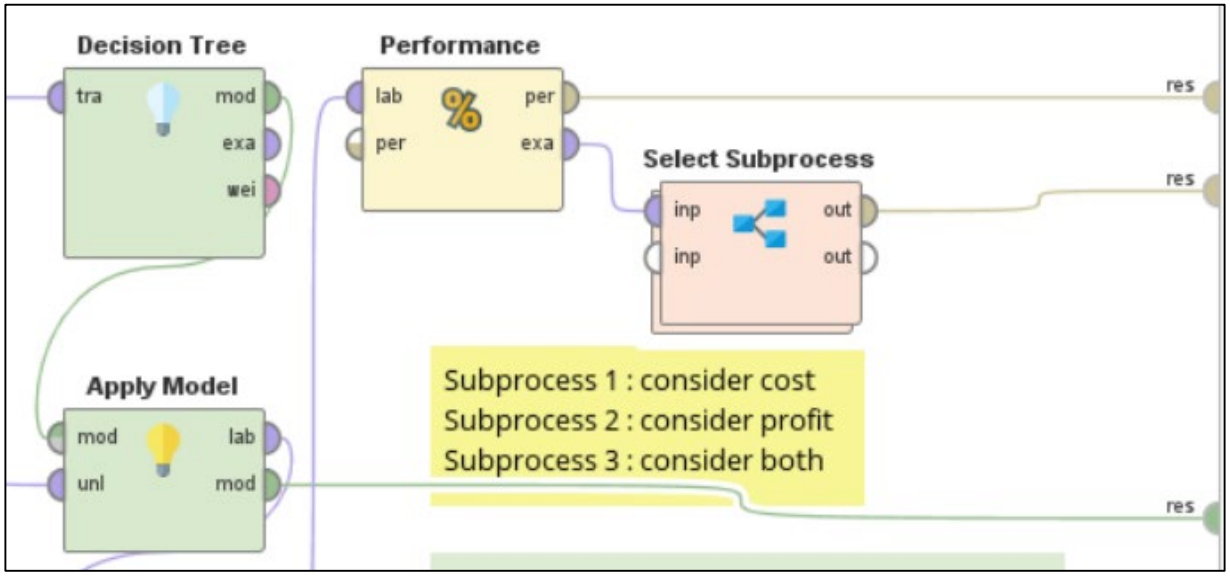
```
----- Classifier 1 -----
              precision    recall  f1-score   support

         bad     0.5050    0.5667    0.5340        90
        good     0.8040    0.7619    0.7824       210

    accuracy                         0.7033       300
   macro avg     0.6545    0.6643    0.6582       300
weighted avg     0.7143    0.7033    0.7079       300

----- Classifier 2 -----
              precision    recall  f1-score   support

         bad     0.5926    0.3556    0.4444        90
        good     0.7642    0.8952    0.8246       210

    accuracy                         0.7333       300
   macro avg     0.6784    0.6254    0.6345       300
weighted avg     0.7127    0.7333    0.7105       300
```

- Comparing f1-scores of class good ➔ classifier 2 is better
- Comparing f1-scores of class bad ➔ classifier 1 is better

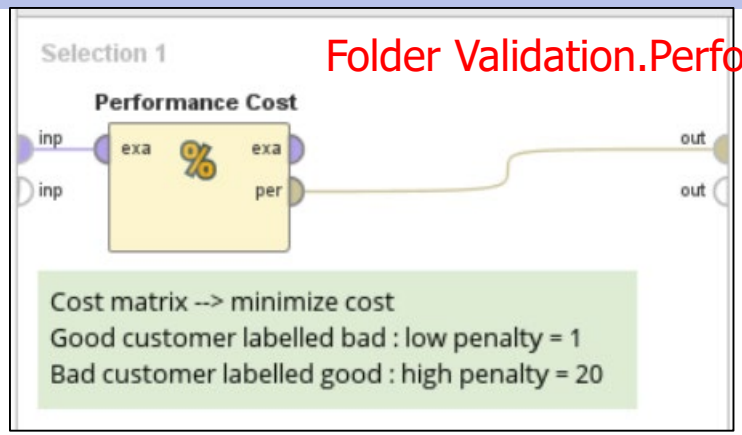
- Averaging performance from both classes
  - Precision, recall, F-measure of each class is weighted by relative size i.e. 90/300 for class bad, 210/300 for class good
  - So classifier 1's weighted avg F-measure

    = (90/300)(0.5340) + (210/300)(0.7824) = 0.7079
- Comparing weighted avg metrics
  - Classifier 2 is generally better (better recall, better F-measure), but it is very biased toward class good
  - If predicting bad is much more important than predicting good, then classifier 1 is better

- Determining which classifier is better depends on applications and the purposes of data analysis
  - Regardless of cost/profit ➔ do we want best overall model, or best model for a certain class ?
  - With respect to cost/profit ➔ different cost/profit matrices may give different conclusions



Consider only Cost



Folder Validation.Performance.Predictive

Cost Matrix **As Cost**	True Class 1 Good	True Class 2 Bad
Predicted Class 1 Good	0.0	20.0
Predicted Class 2 Bad	1.0	0.0

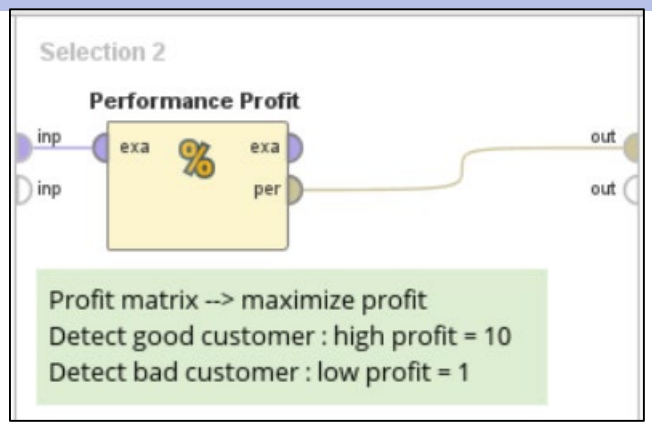
Misclassificationcosts
Classifier 1
Misclassificationcosts: 2.767

Misclassificationcosts
Classifier 2
Misclassificationcosts: 3.940

Classifier 1 total cost = (39*20) + (50*1) = 830 avg cost = 830/300 = 2.767
Classifier 2 total cost = (58*20) + (22*1) = 1182 avg cost = 1182/300 = 3.94

Classifier 1 gives lower average cost (of wrong prediction) ➔ better at avoiding mistaking bad customers as good ones

Consider only Profit



Cost Matrix **As Profit**	True Class 1 Good	True Class 2 Bad
Predicted Class 1 Good	10.0	0.0
Predicted Class 2 Bad	0.0	1.0

Misclassificationcosts
Classifier 1
`Misclassificationcosts: 5.503`

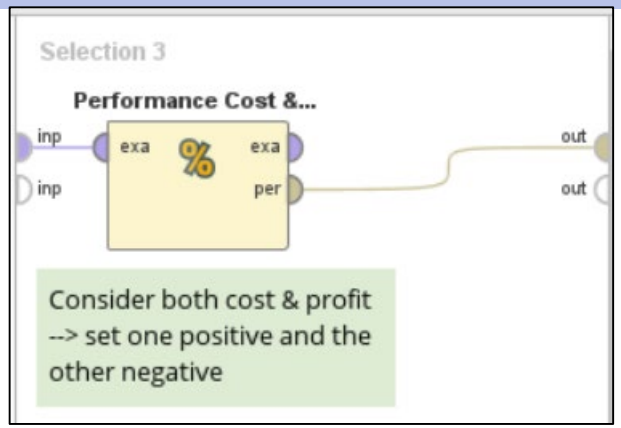
Misclassificationcosts
Classifier 2
`Misclassificationcosts: 6.373`

Interpreted as profit

Calculation is done in the same way as the calculation of total & avg. cost

Classifier 2 gives higher average profit (of correct prediction) ➔ better at picking good customers

Consider both Cost and Profit



Cost Matrix **As Cost**	True Class 1 Good	True Class 2 Bad
Predicted Class 1 Good	-10.0	20.0
Predicted Class 2 Bad	1.0	-1.0

Misclassificationcosts
Classifier 1
`Misclassificationcosts: -2.737`

Misclassificationcosts
Classifier 2
`Misclassificationcosts: -2.433`

We set positive value for cost & negative value for profit

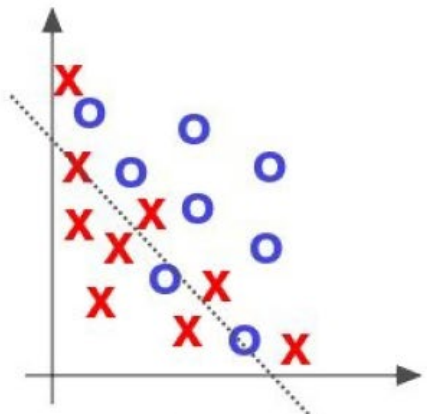
Classifier 1 gives higher profit

Training vs. Generalization Performance

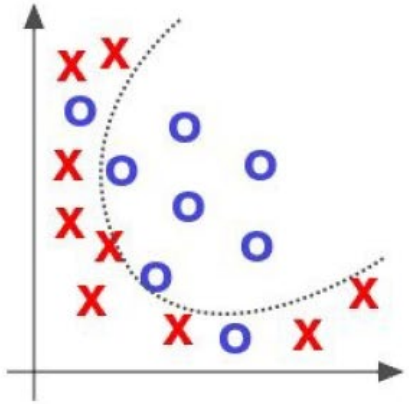
- Training performance
  - Calculated in training phase by applying the model to training data
  - Help determine, for example,
    - Whether to add/remove rule conditions
    - Whether to continue one more training cycle of neural network
- Generalization (or validation or testing) performance
  - Calculated in prediction phase by applying the model to testing data
  - Measure how the model generalizes to unseen data
  - Slight decrease from training performance is expected
- By comparing training and generalization performance, we can detect model underfitting or overfitting

Underfitting vs. Overfitting

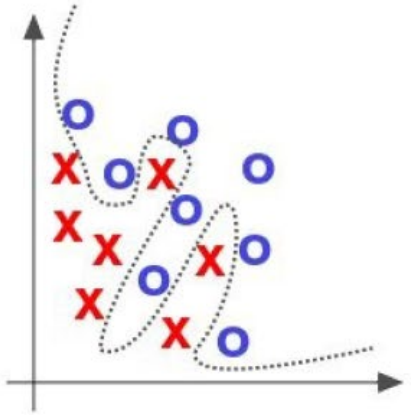
- Underfitting
  - The model is too simple. It fails to learn the true structure of data
  - High training error, high generalization error
    (bad performance in both training and testing)
- Overfitting
  - The model is too complicated. It tries to fit the training data too much but cannot generalize to unseen data
  - Low training error, high generalization error
    (good training performance, much worse testing performance)
  - Possible reasons
    - The model tries to classify noises
    - Lack of representative examples. Some characteristics are missing from training data



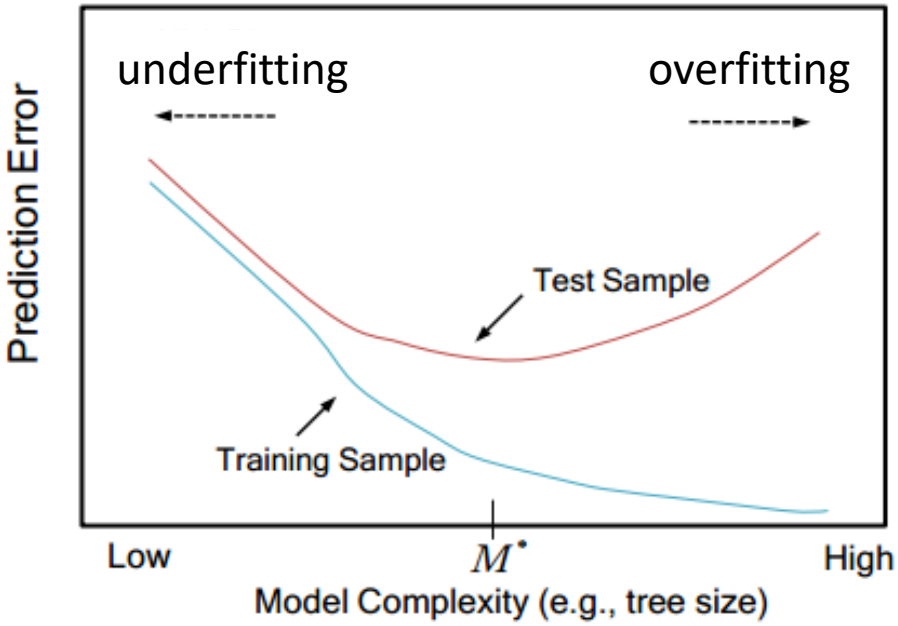
Underfitting



Good fitting

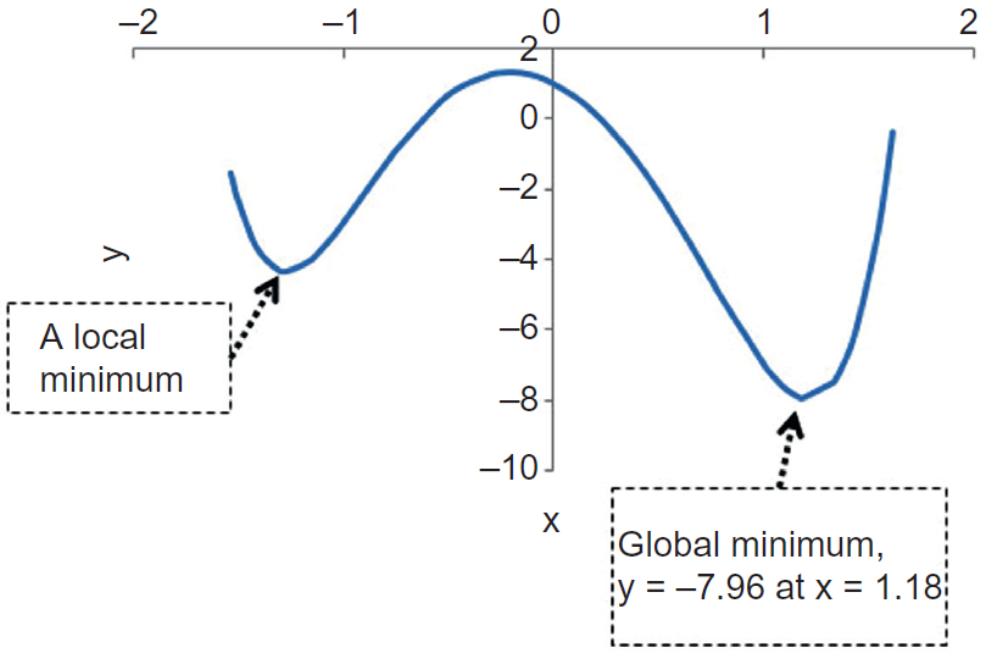


Overfitting

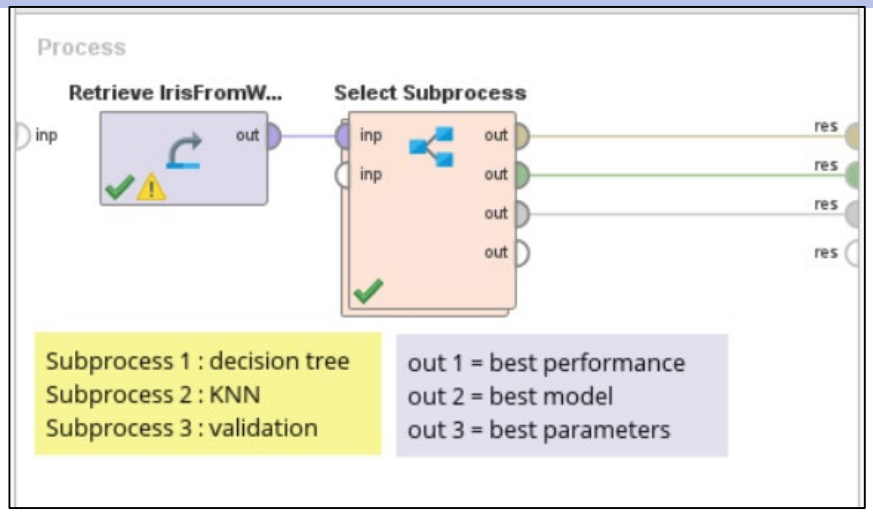


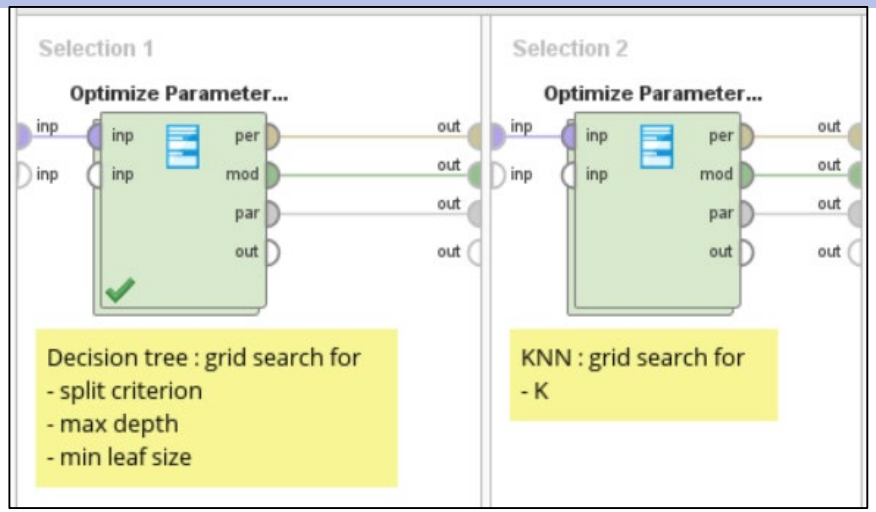
Optimization Grid

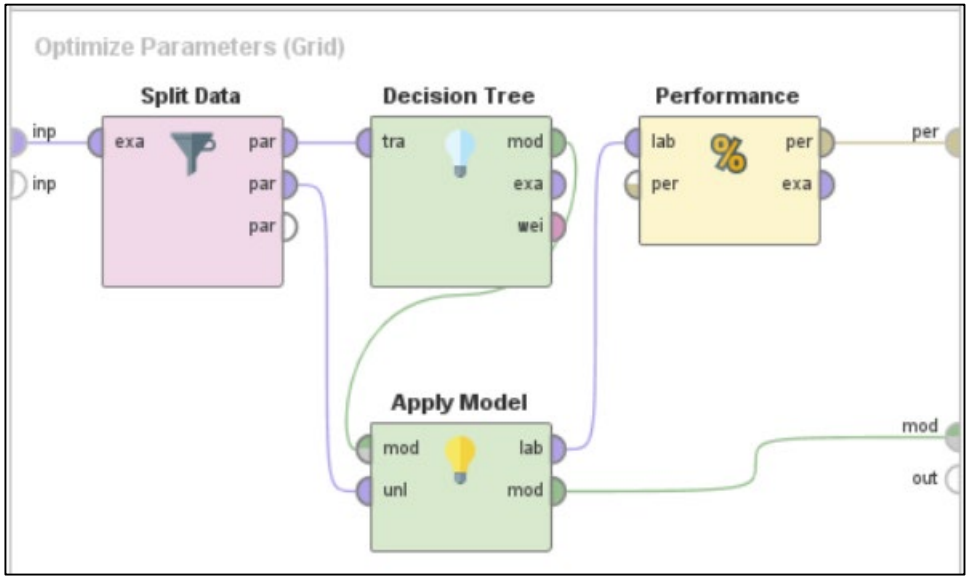
- Complicated models tend to have many parameters
  - Default params are good starting points but not always the best ones
  - Manual adjustment is difficult. Good performance that we get may only be a local optimal point



Example (2) : optimization grid

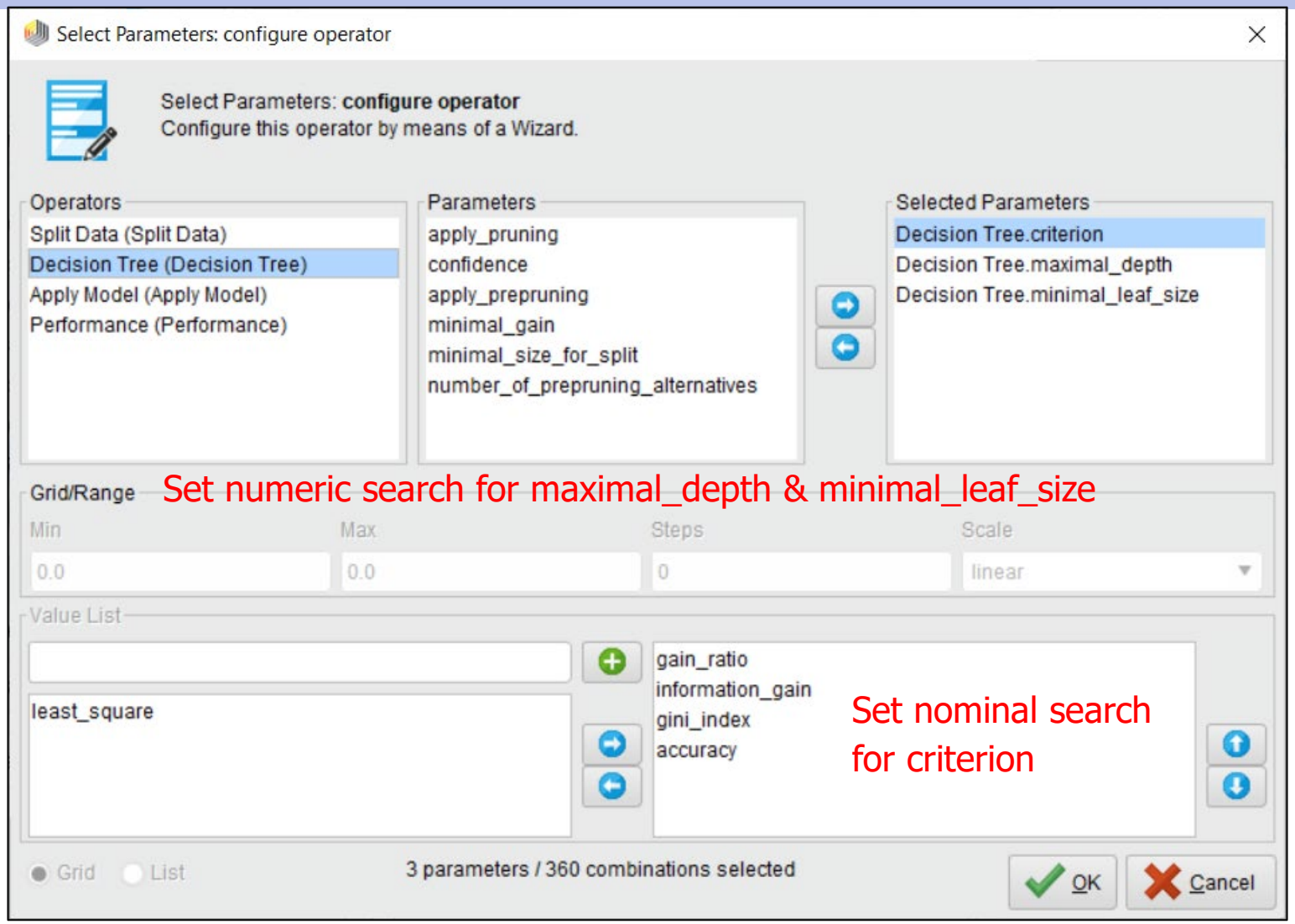


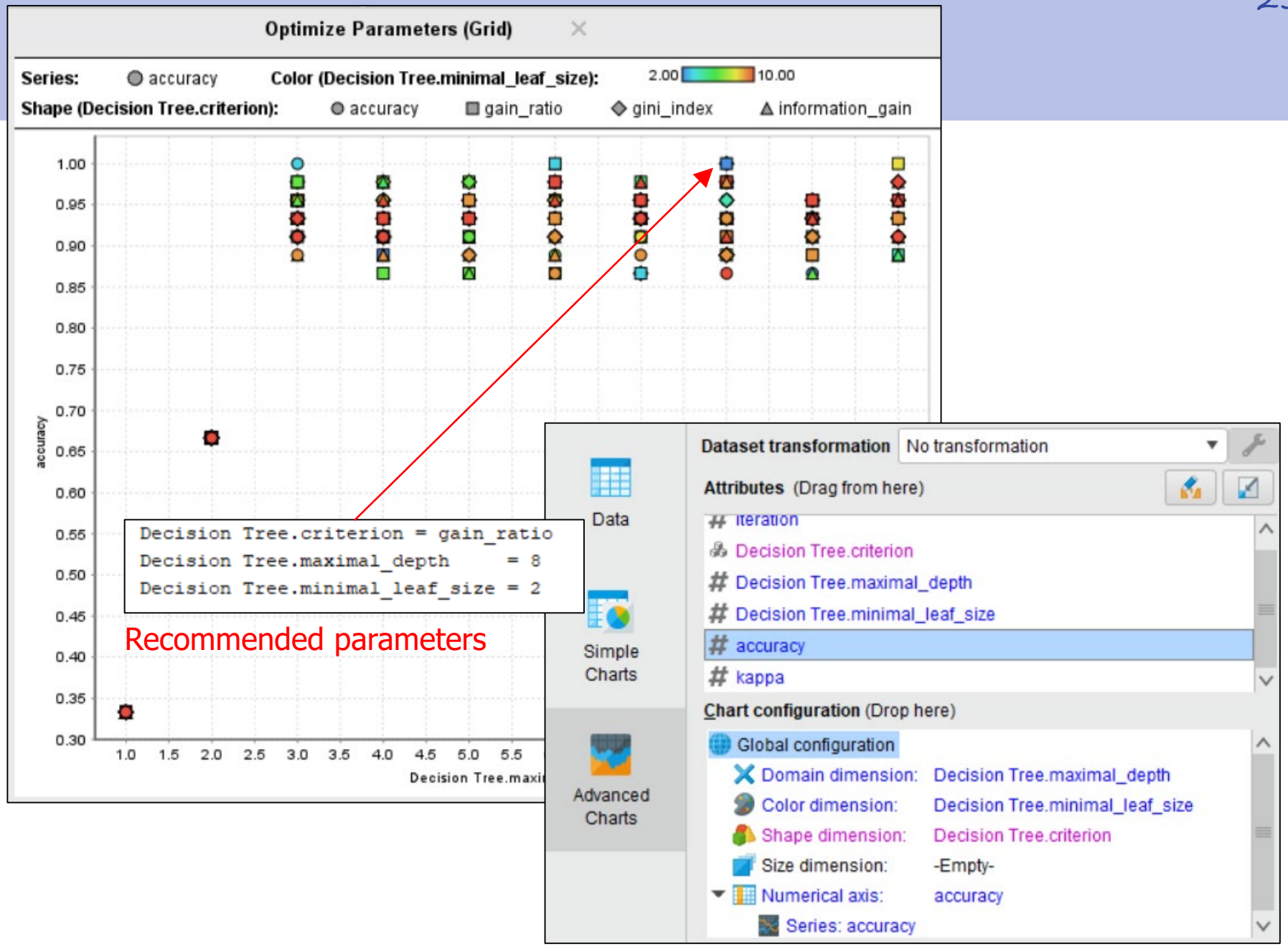




Can be changed to "Validation" operators as in Examples 3, 4, 5

Alternatively, we can play with "Optimization + Split" operators, to get optimal parameters. Then use "Validation" operator to get formal performance statistics


Select Parameters: configure operator
Select Parameters: configure operator
Configure this operator by means of a Wizard.
Operators
Split Data (Split Data)
Decision Tree (Decision Tree)
Apply Model (Apply Model)
Performance (Performance)
Parameters
apply_pruning
confidence
apply_prepruning
minimal_gain
minimal_size_for_split
number_of_prepruning_alternatives
Selected Parameters
Decision Tree.criterion
Decision Tree.maximal_depth
Decision Tree.minimal_leaf_size
Grid/Range
Set numeric search for maximal_depth & minimal_leaf_size
Min
Max
Steps
Scale
0.0
0.0
0
linear
Value List
least_square
gain_ratio
information_gain
gini_index
accuracy
Set nominal search for criterion
Grid
List
3 parameters / 360 combinations selected
OK
Cancel


Optimize Parameters (Grid)
Series: accuracy
Color (Decision Tree.minimal_leaf_size): 2.00 10.00
Shape (Decision Tree.criterion): accuracy gain_ratio gini_index information_gain
accuracy
Decision Tree.maxi
Decision Tree.criterion = gain_ratio
Decision Tree.maximal_depth = 8
Decision Tree.minimal_leaf_size = 2
Recommended parameters
Data
Simple Charts
Advanced Charts
Dataset transformation
No transformation
Attributes (Drag from here)
Iteration
Decision Tree.criterion
Decision Tree.maximal_depth
Decision Tree.minimal_leaf_size
accuracy
kappa
Chart configuration (Drop here)
Global configuration
Domain dimension: Decision Tree.maximal_depth
Color dimension: Decision Tree.minimal_leaf_size
Shape dimension: Decision Tree.criterion
Size dimension: -Empty-
Numerical axis: accuracy
Series: accuracy

Validation Methods

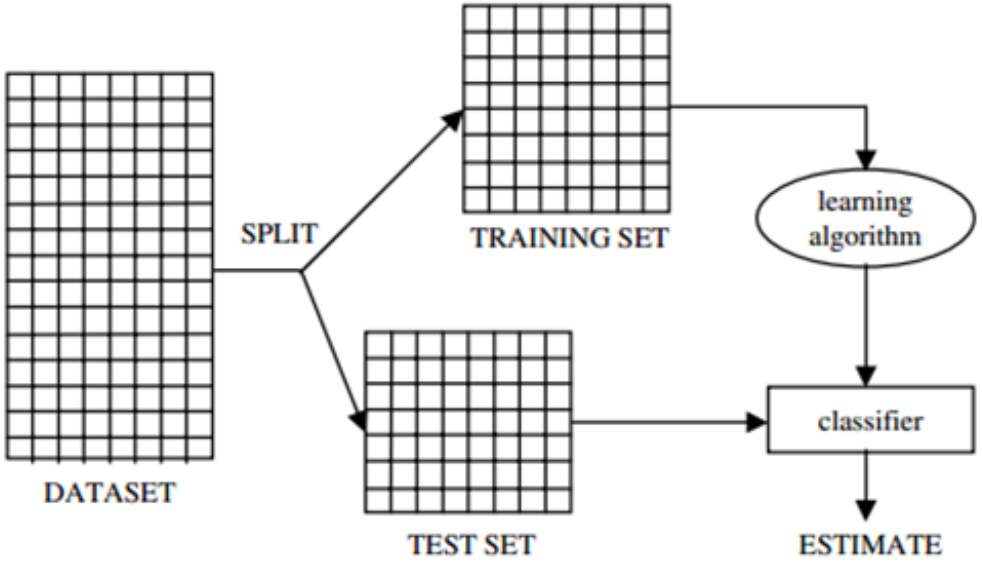
To prepare training and testing data

- Simple approach = use separate data sets for training and testing
  - Both data sets must have the same structures (attributes and types)
- Split available data into training and testing
  - Split validation or holdout
  - Cross validation
  - Bootstrapping validation

  } Classifier is evaluated many times, so more performance statistics can be calculated

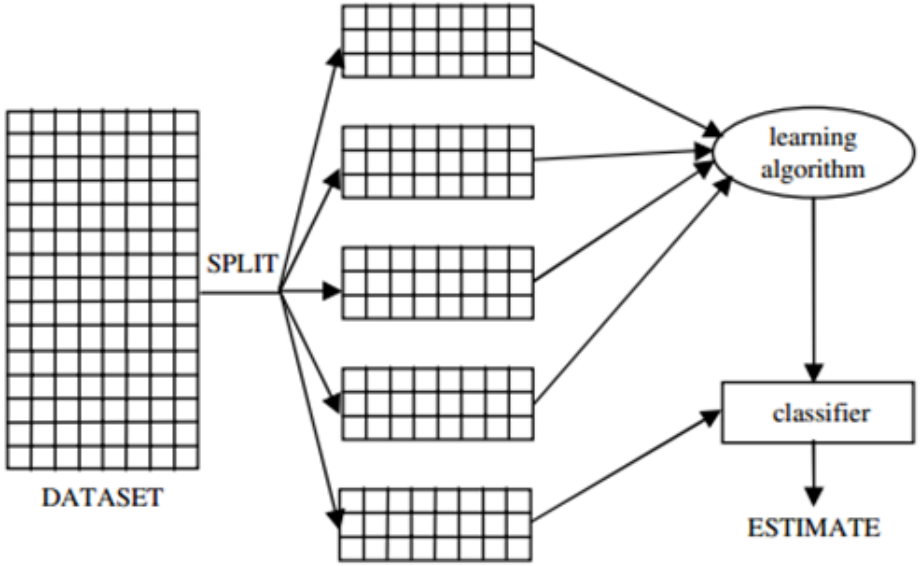
  - After the model is evaluated on testing data, the final working model can be built from the full set of data (training + testing) ➔ the more data, the better

- **Split validation (holdout)**
  - Split data into 2 disjoint sets, each contains representative samples
  - The proportion of training & testing sets
    - Small training set leads to underfitting model
    - Small testing set leads to unreliable performance evaluation
    - Suggested proportion is 2:1 (66% for training, 34% for testing)



Cross validation

- Split data into k sets and build k classifiers. For each classifier
  - One set is used as testing data
  - The remaining k-1 sets are used as training data
- Average the performance from k classifiers
- Common practice is 10-fold cross validation

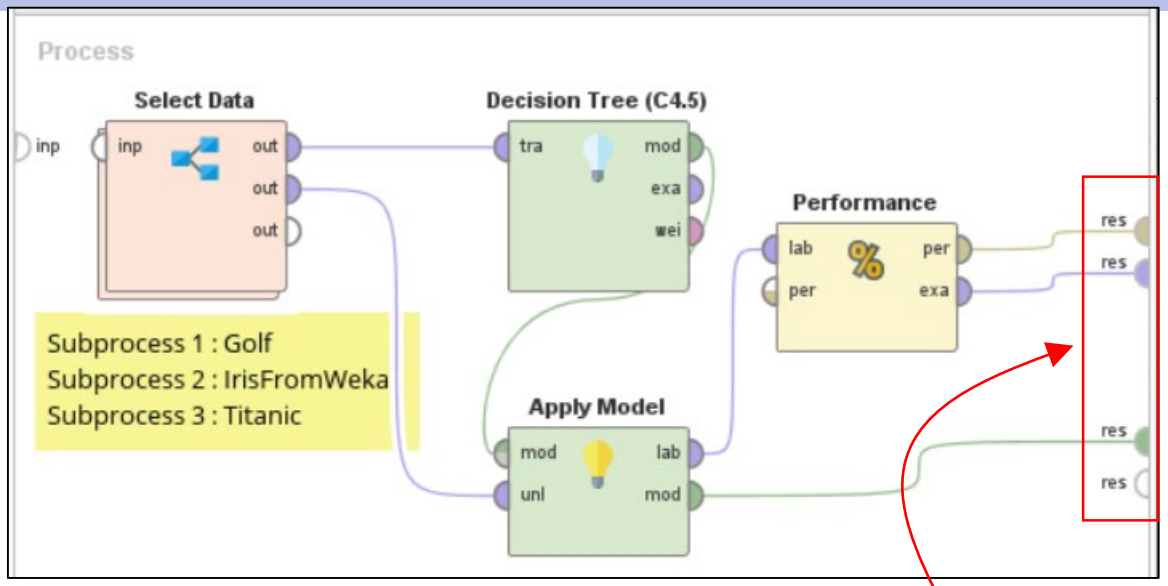


Each record is used for both training & testing, thus reducing the effect of input sensitivity

More statistics can be calculated e.g. SD of performance estimation

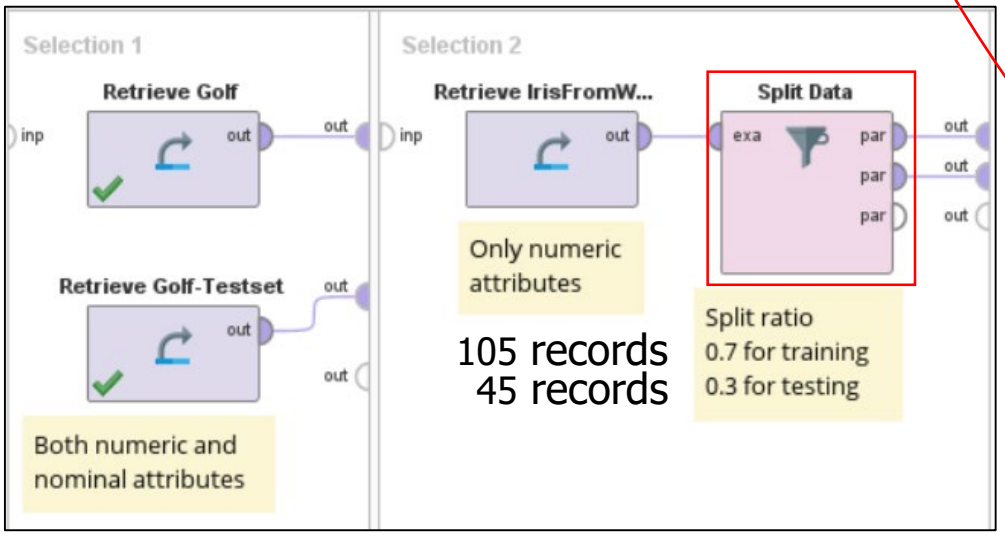
- **Bootstrapping validation**
  - Sampling with replacement n times
    - All chosen records are used as training data
    - The remaining records are used as testing data
  - Good for small data set
  - But may result in overfitting model because some records are chosen and processed many times as training data
    - Can be alleviated by repeating bootstrapping validation m times
    - Average the performance from m validations
    - Other statistics such as SD can also be calculated

Decision tree example in Chapter 4 (Ex 4.1)



Performance is based on 45 testing records

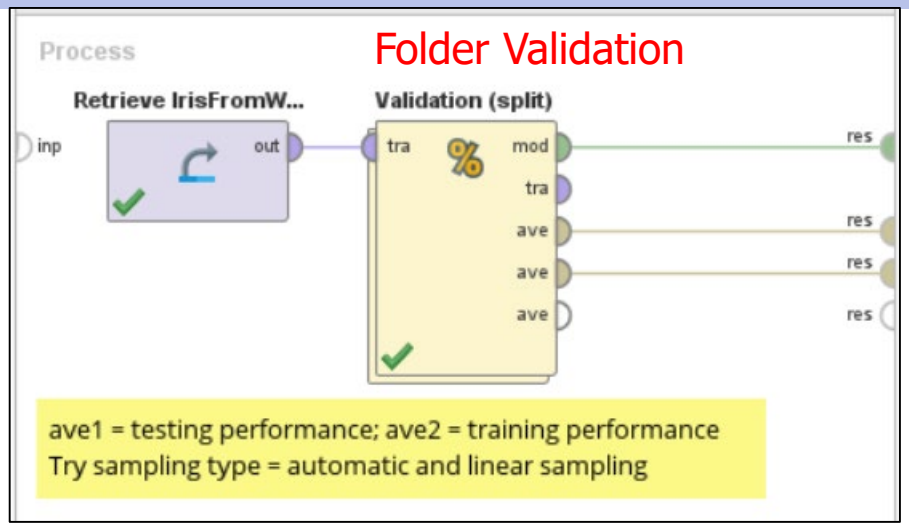
Model is built from 105 training records



This approach is good for quick testing of methods & params

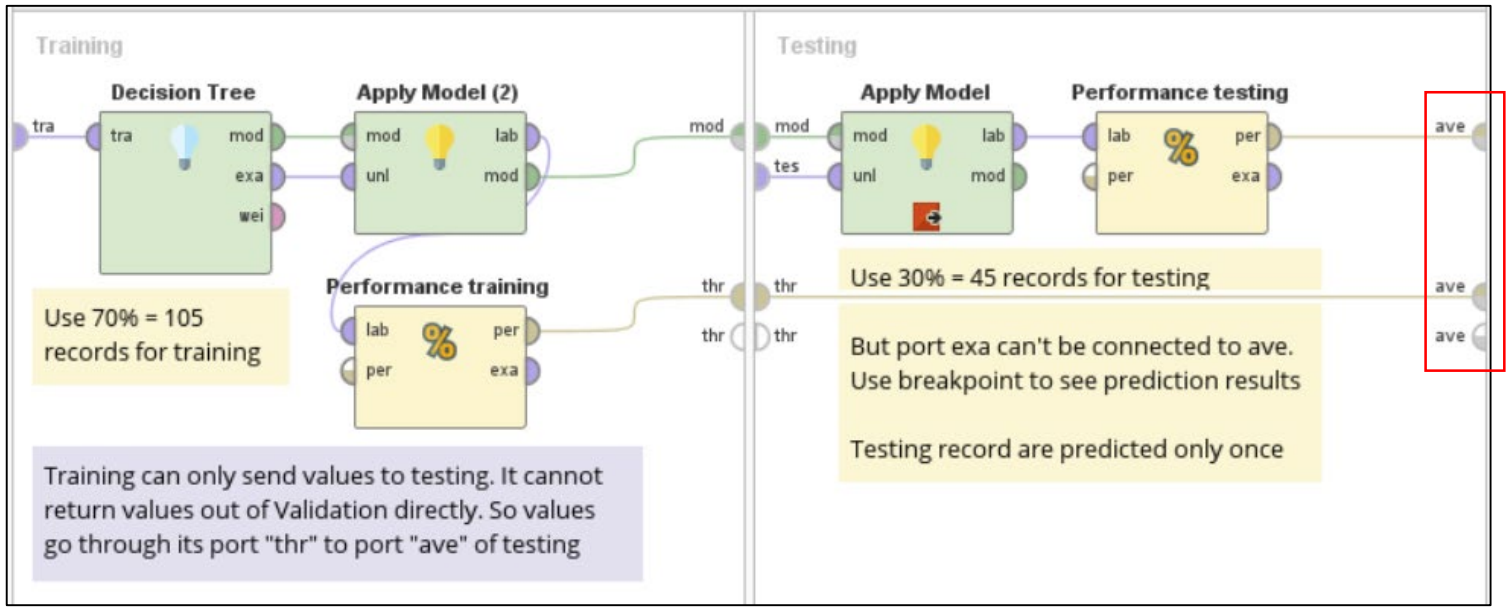
"res" port is more flexible than "ave" port in Validation operator because it can send any type of results to Result buffer

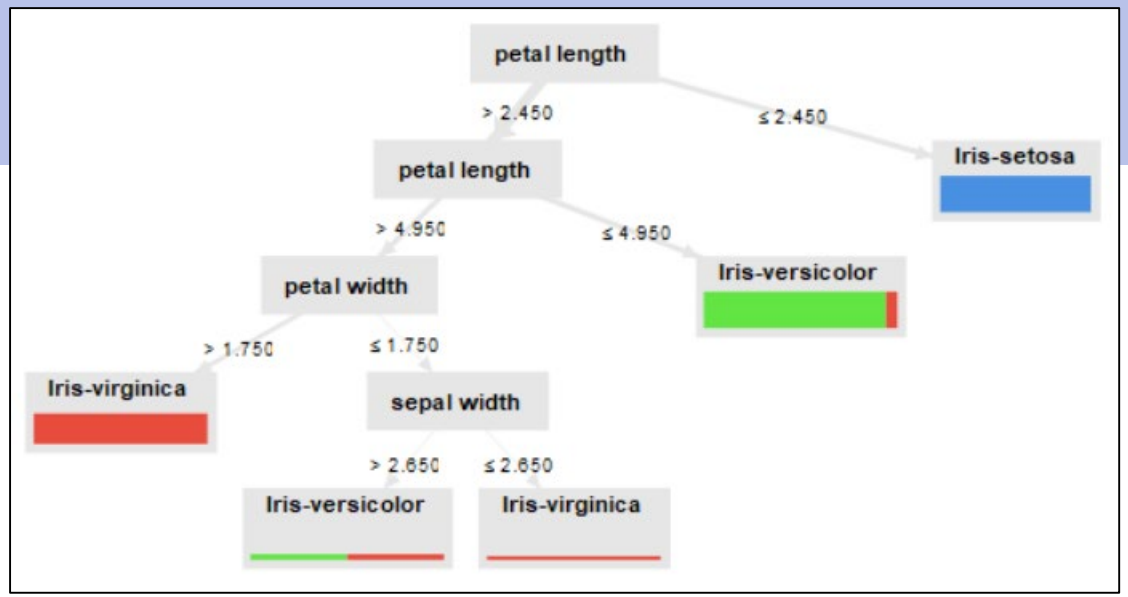
Example (3) : split validation



Final model is built from all 150 records

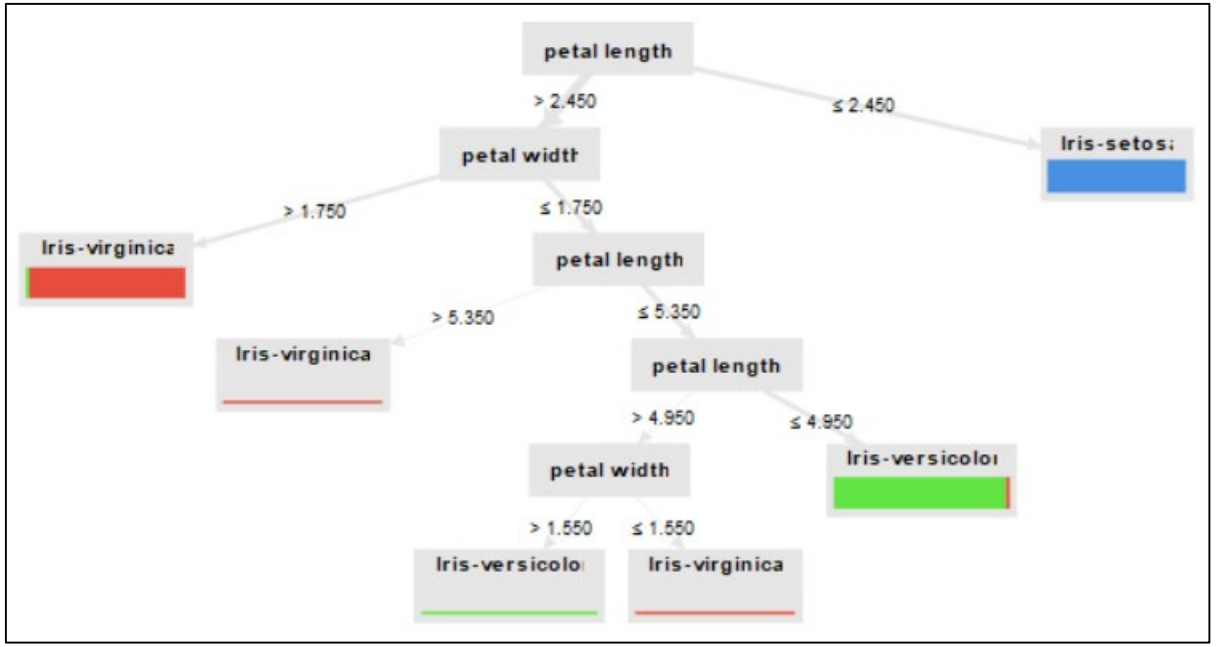
In split validation, prediction on testing data is done only once





In Ex 4.1, tree is built from training data

In this example, tree is built from whole data set



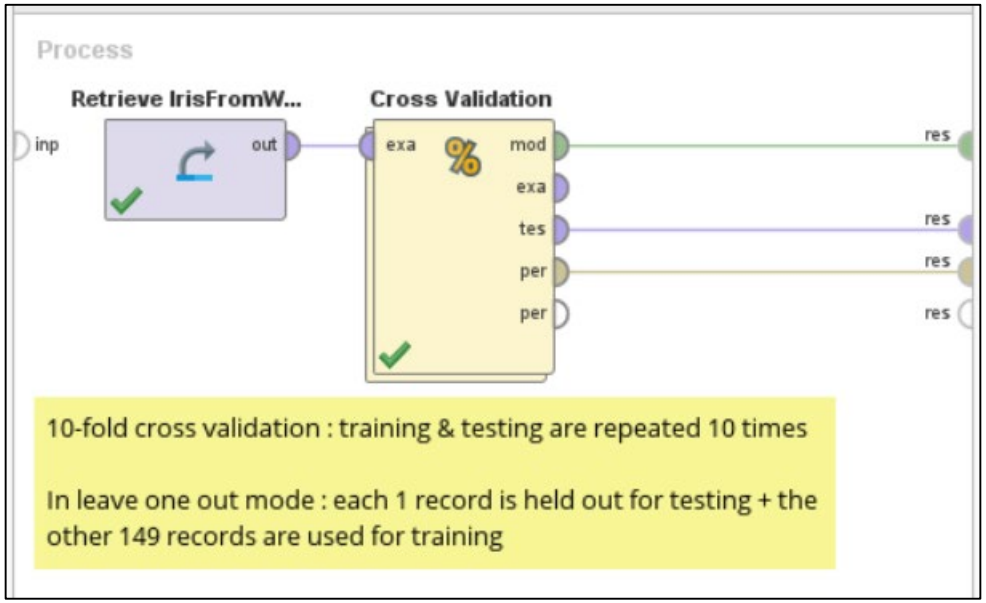
The model can be slightly different because it is built from different (bigger) set of data

But conditions for majority of Setosa, Versicolor, and Virginica remain the same

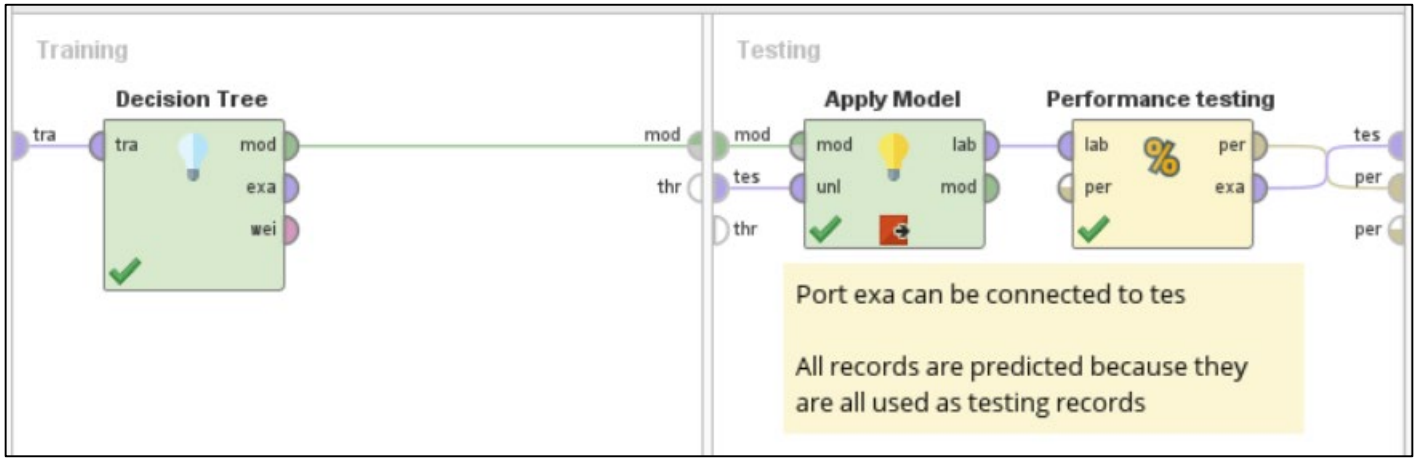
What does overfitting look like ?

- Try validation parameter : sampling type = automatic ➔ training & testing records are sampled
  - Training accuracy = 96.19%, testing accuracy = 91.11%
  - All precision & recall values are close
  - The model can generalize well to unseen data
- Try validation parameter : sampling type = linear sampling ➔ the first 105 records are for training, the rest are for testing
  - Training accuracy = 99.05%, testing accuracy = 57.78%
  - Testing performance is much worse
  - The model overfits training data but cannot generalize to unseen data
    - Training records = 50 Setosa + 50 Versicolor + 5 Virginica
    - Testing records = 45 Virginica

Example (4) : cross validation



Prediction on testing data is done 10 times and 10 performance metrics are averaged



accuracy: 95.33% +/- 4.50% (micro average: 95.33%)

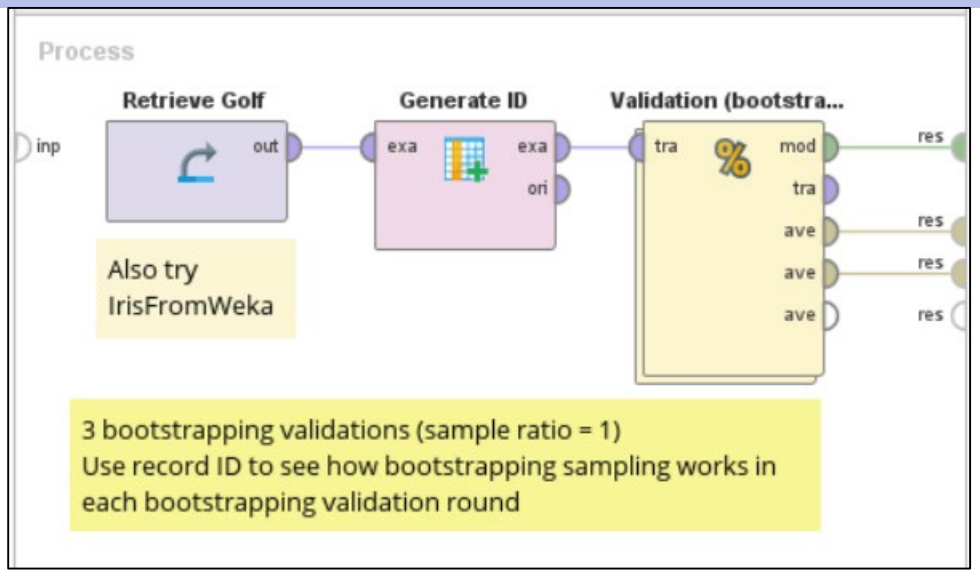
	true Iris-setosa	true Iris-versicolor	true Iris-virginica	class precision
pred. Iris-setosa	50	0	0	100.00%
pred. Iris-versicolor	0	46	3	93.88%
pred. Iris-virginica	0	4	47	92.16%
class recall	100.00%	92.00%	94.00%	

Average accuracy (from 10 folds) = 95.33% with SD = 4.5%
In leave-one-out mode, SD will be much higher because the performance is averaged from 150 folds

$$\text{(Macro) average accuracy} = \frac{\text{sum of 10 accuracies}}{10}$$

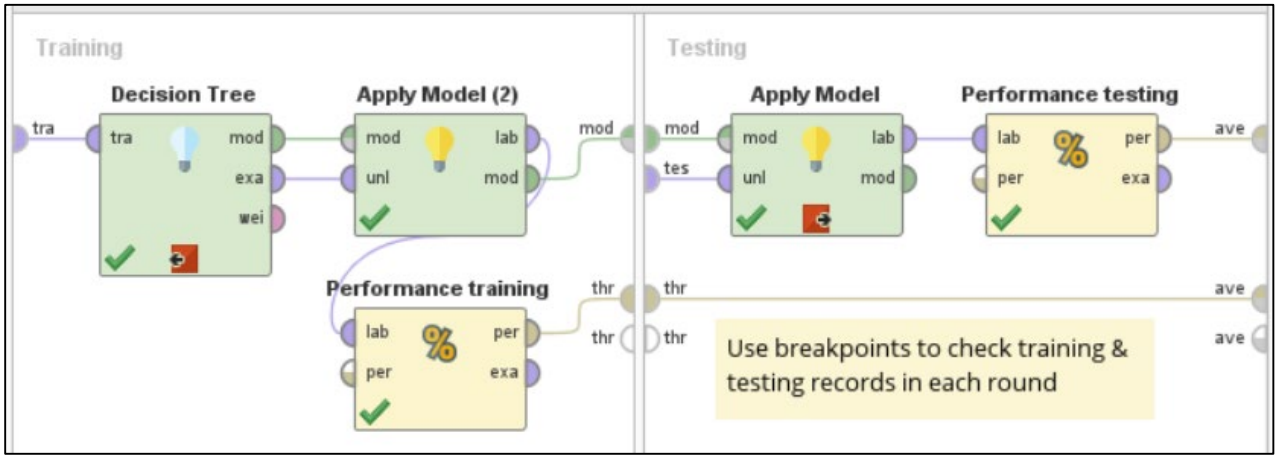
$$\text{Micro average accuracy} = \frac{\text{sum of correctly predicted records in 10 runs}}{\text{sum of testing records in 10 runs}}$$

Example (5) : bootstrapping validation



Prediction on testing data is done 3 times and 3 performance metrics are averaged

In each round ➔ sampling with replacement 14 times
All chosen records for training
Never chosen records for testing



RapidMiner builds final model from full data. But difference between training & testing performance makes it look like an overfitting model

Averaging performance metrics

- Consider a performance metric of 3 classes $pfm(A), pfm(B), pfm(C)$
- **Macro average**

$$= \frac{pfm(A) + pfm(B) + pfm(C)}{3}$$

  - All classes contribute equally, so small classes are over-emphasized
  - Easy calculation

- **Micro average**

Put TP, TN, FP, FN from all classes in to a single matrix and calculate pfm

$$\text{Micro average recall} = \frac{\text{Total } TP \text{ from all classes}}{\text{Total actual } (TP + FN) \text{ from all classes}}$$

$$\text{Micro average precision} = \frac{\text{Total } TP \text{ from all classes}}{\text{Total predict } (TP + FP) \text{ from all classes}}$$

  - All records contribute equally, so big classes (more records) contribute more
  - Need to check TP, TN, FP, FN of all classes ➔ more calculation overhead

- **Weighted average**

$= W_A\, pfm(A) + W_B\, pfm(B) + W_C\, pfm(C) \quad ; W = \text{relative size of each class}$

  - A compromise between macro and micro average
  - Calculation is easier than micro average when performance metrics of each class are already available

Example : imbalanced classes (1)

		Actual		Total
		A	B	
Predict	A	750	150	900
	B	50	50	100
Total		800	200	1000

Class	TP	FP	FN	Precision	Recall
A	750	150	50	750/900 = 0.83	750/800 = 0.94
B	50	50	150	50/100 = 0.5	50/200 = 0.25

When big class (i.e. class A) performs very well

- Macro average recall = (0.94 + 0.25) / 2 = 0.595
- Micro average recall = (750+50) / (800+200) = 0.8
- Weighted average recall = (0.8*0.94) + (0.2*0.25) = 0.802

- Micro or weighted avg is preferred because macro avg is too low
- But if small class is more important, high micro & weighted avg can be misleading. Macro avg is preferred because it is better at telling that the classifier's avg recall still isn't good

Example : imbalanced classes (2)

		Actual		Total
		A	B	
Predict	A	50	50	100
	B	750	150	900
Total		800	200	1000

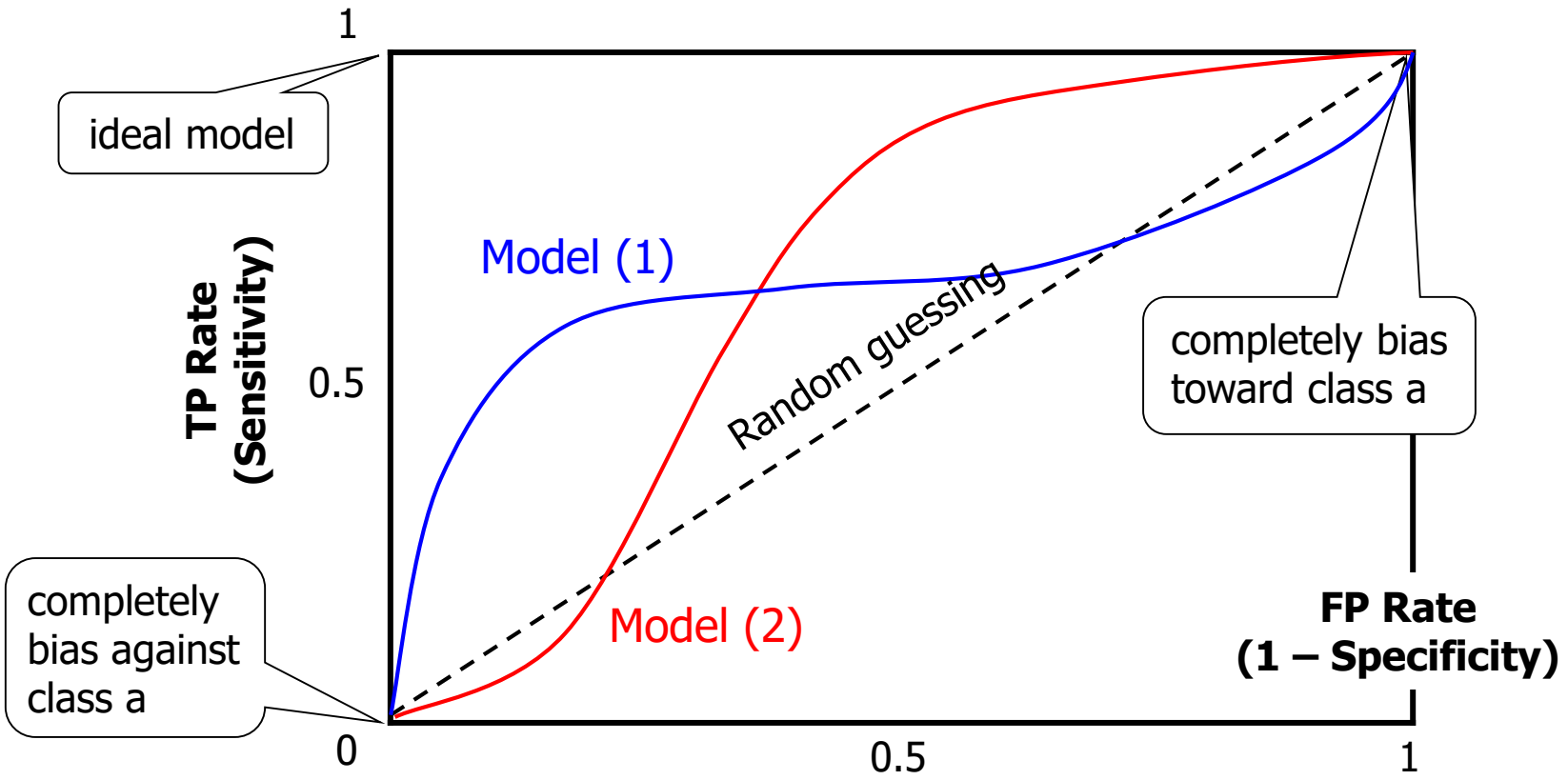
Class	TP	FP	FN	Precision	Recall
A	50	50	750	50/100 = 0.5	50/800 = 0.06
B	150	750	50	150/900 = 0.17	150/200 = 0.75

When big class (i.e. class A) performs badly

- Macro average recall = (0.06 + 0.75) / 2 = 0.405
- Micro average recall = (50+150) / (800+200) = 0.2
- Weighted average recall = (0.8*0.06) + (0.2*0.75) = 0.198

- Micro or weighted avg is preferred because macro avg is too high
- But if small class is more important, low micro & weighted avg can be misleading. Macro avg is preferred because it is better at telling that the classifier's avg recall is moderate

Trade-off Between TP Rate and FP Rate



ROC (Receiver Operating Characteristic) curve

Interpreting ROC Curve

- Trade-off between positive hits (TP) and false alarms (FP)
- Diagonal line implies random guessing
  - Above diagonal = good model
  - Below diagonal = bad model (predict the opposite of true class)
  - From the example, none is clearly better than the other
    - At small FP rate, model (1) is better
    - At large FP rate, model (2) is better
  - So we compute the area under ROC curve (AUC)
    - Perfect model : area = 1
    - Random guessing : area = 0.5
    - Model with bigger area is better on average

Plotting ROC Curve

Positive class (P) = class being analyzed

Negative class (N) = other classes

1. For each testing record X, calculate confidence or prob. that X is positive
2. Sort all testing records in decreasing order of their prob. In case that >1 records have equal prob.
   - Optimistic ROC — correct prediction (actual class = P) comes first
   - Pessimistic ROC — incorrect prediction comes first
   - Neutral ROC — alternate between optimistic and pessimistic
3. For each threshold t
   - If P(X) ≥ t, predict P
   - Calculate TP rate (TPR = TP / actual P) and FP rate (FPR = FP / actual N) for this threshold

X	Actual Class	Prob. of predicting "P"	TP	FP	TN	FN	TPR = TP/actual P	FPR = FP/actual N
1	P	0.9	1	0	5	4	1/5 = 0.2	0/5 = 0
2	P	0.8						
3	N	0.7						
4	P	0.6						
5	P	0.55						
6	N	0.54						
7	N	0.53						
8	N	0.51						
9	P	0.5						
10	N	0.4						

*Use each prob. as **t**, e.g. when **t = 0.9***
Record 1 is predicted "P"
The others are predicted "N"

- ★ *TP records = {1}*
- ★ *FP records = ∅*
- ★ *TN records = {3, 6, 7, 8, 10}*
- ★ *FN records = {2, 4, 5, 9}*

X	Actual Class	Prob. of predicting "P"	TP	FP	TN	FN	TPR = TP/actual P	FPR = FP/actual N
1	P	0.9	1	0	5	4	1/5 = 0.2	0/5 = 0
2	P	0.8	2	0	5	3	2/5 = 0.4	0/5 = 0
3	N	0.7	2	1	4	3	2/5 = 0.4	1/5 = 0.2
4	P	0.6						
5	P	0.55						
6	N	0.54						
7	N	0.53						
8	N	0.51						
9	P	0.5						
10	N	0.4						

*When **t = 0.7***

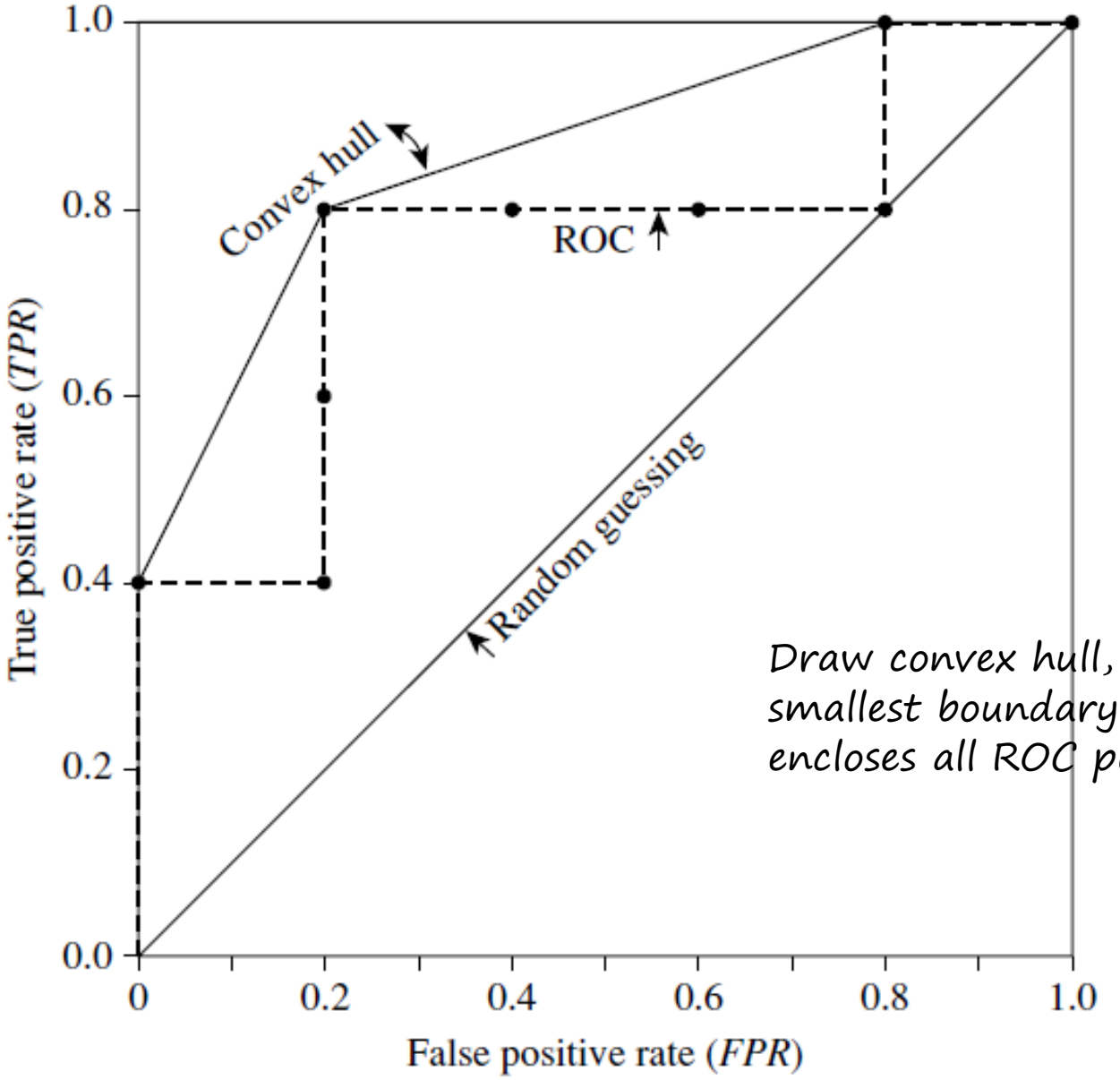
{1, 2, 3} are predicted "P"

The others are predicted "N"

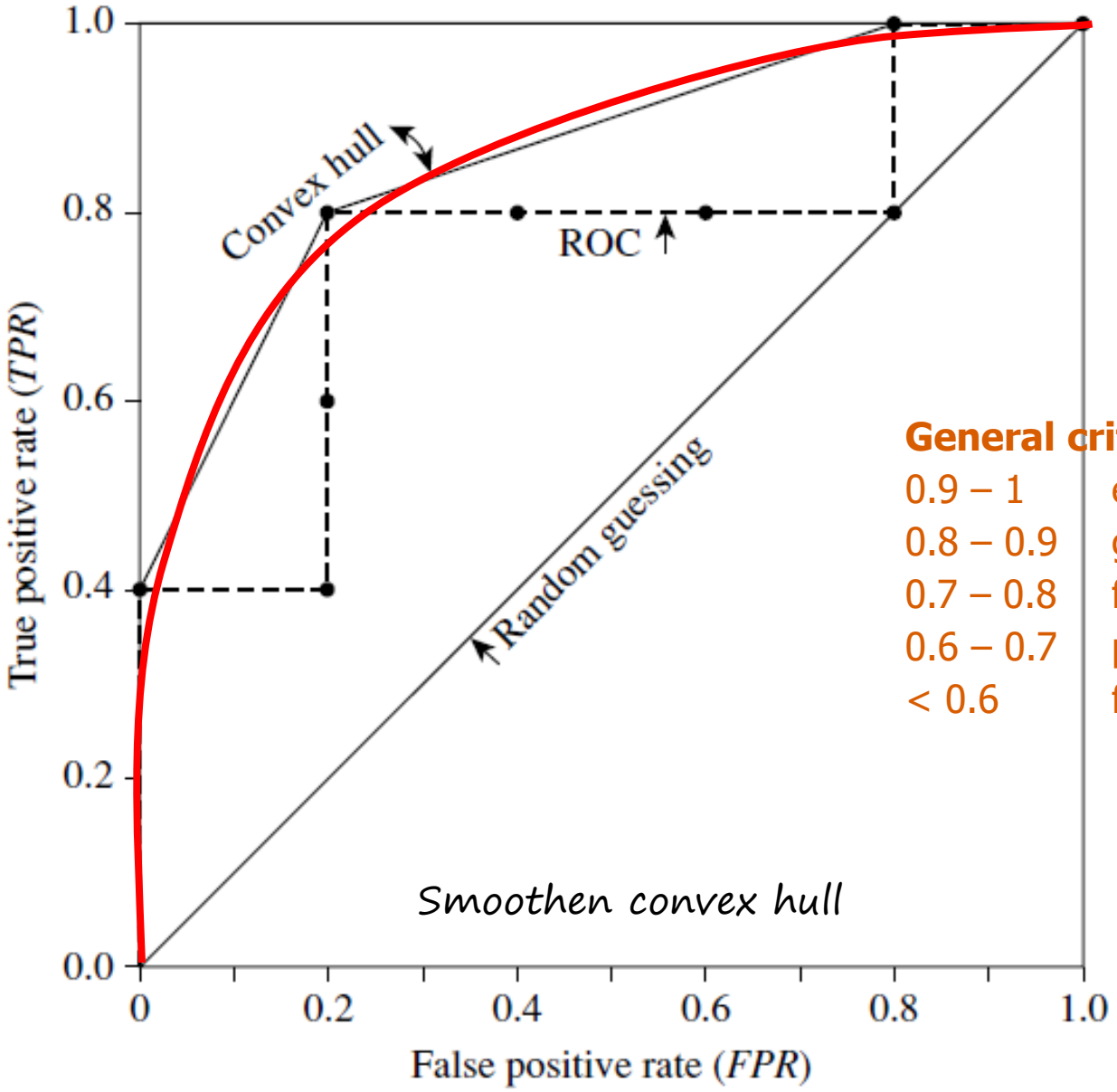
- ★ *TP records = {1, 2}*
- ★ *FP records = {3}*
- ★ *TN records = {6, 7, 8, 10}*
- ★ *FN records = {4, 5, 9}*

X	Actual Class	Prob. of predicting "P"	TP	FP	TN	FN	TPR = TP/actual P	FPR = FP/actual N
1	P	0.9	1	0	5	4	1/5 = 0.2	0/5 = 0
2	P	0.8	2	0	5	3	2/5 = 0.4	0/5 = 0
3	N	0.7	2	1	4	3	2/5 = 0.4	1/5 = 0.2
4	P	0.6	3	1	4	2	3/5 = 0.6	1/5 = 0.2
5	P	0.55	4	1	4	1	4/5 = 0.8	1/5 = 0.2
6	N	0.54	4	2	3	1	4/5 = 0.8	2/5 = 0.4
7	N	0.53	4	3	2	1	4/5 = 0.8	3/5 = 0.6
8	N	0.51	4	4	1	1	4/5 = 0.8	4/5 = 0.8
9	P	0.5	5	4	1	0	5/5 = 1	4/5 = 0.8
10	N	0.4	5	5	0	0	5/5 = 1	5/5 = 1

Precision-recall curve (PRC) can also be drawn from these values



Draw convex hull, i.e. the smallest boundary that encloses all ROC points

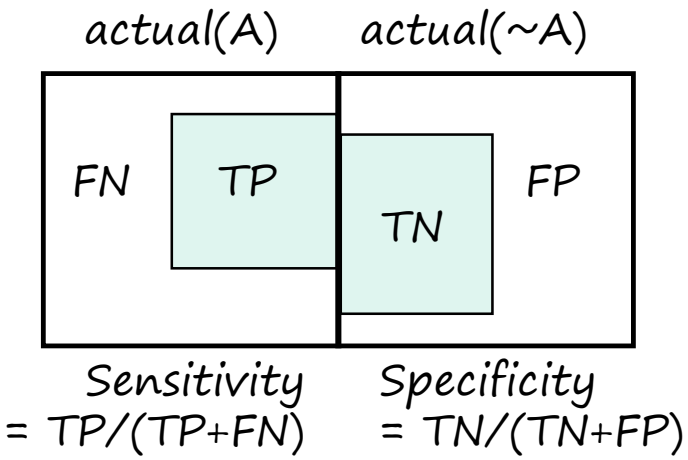


General criteria

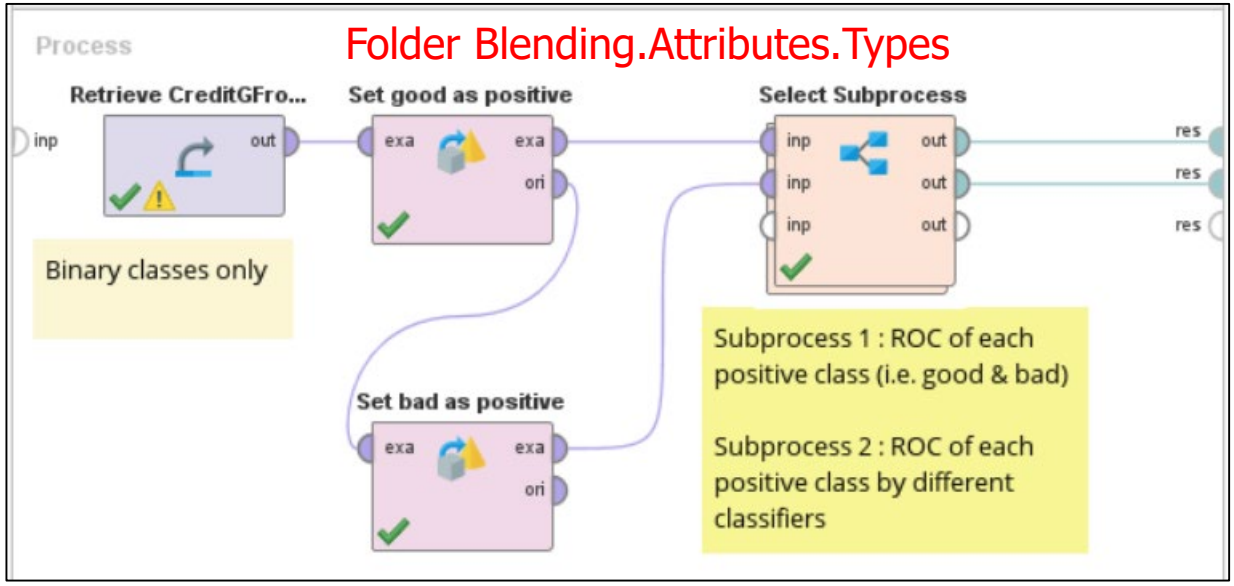
0.9 – 1	excellent classifier
0.8 – 0.9	good classifier
0.7 – 0.8	fair classifier
0.6 – 0.7	poor classifier
< 0.6	failure

More Performance Metrics

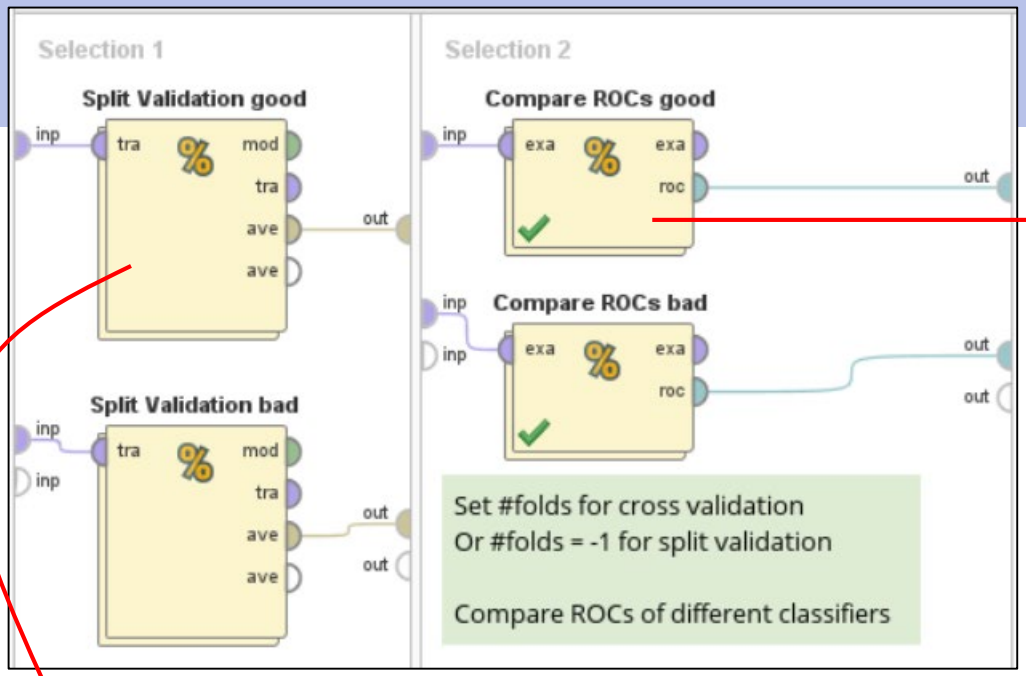
- **Sensitivity (A)** = TP rate or recall (A) = TP(A) / all actual(A)
  - Ability to retrieve what we are looking for, e.g. a certain disease
  - Sensitivity = 1 ➔ the tests are positive for all patients with disease
- **Specificity (A)** = TN rate (A) = TN(A) / all actual(~A)
  - Ability to clear irrelevant cases that we don't want
  - Specificity = 1 ➔ the tests are negative for all patients without disease
- **Youden's J-index (A)** = sensitivity(A) + specificity(A) – 1 ; $0 \leq J \leq 1$
  - J = 1 indicates perfect test
  - J = 0 indicates useless test

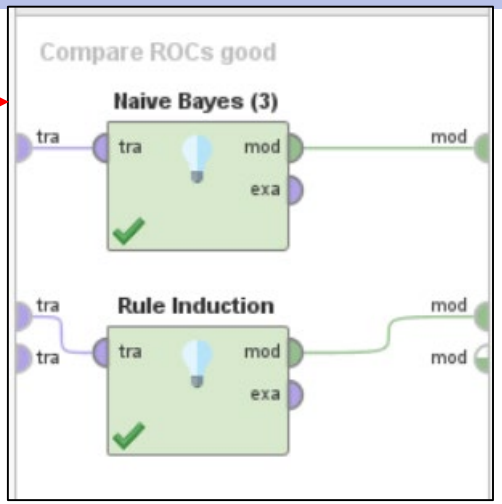


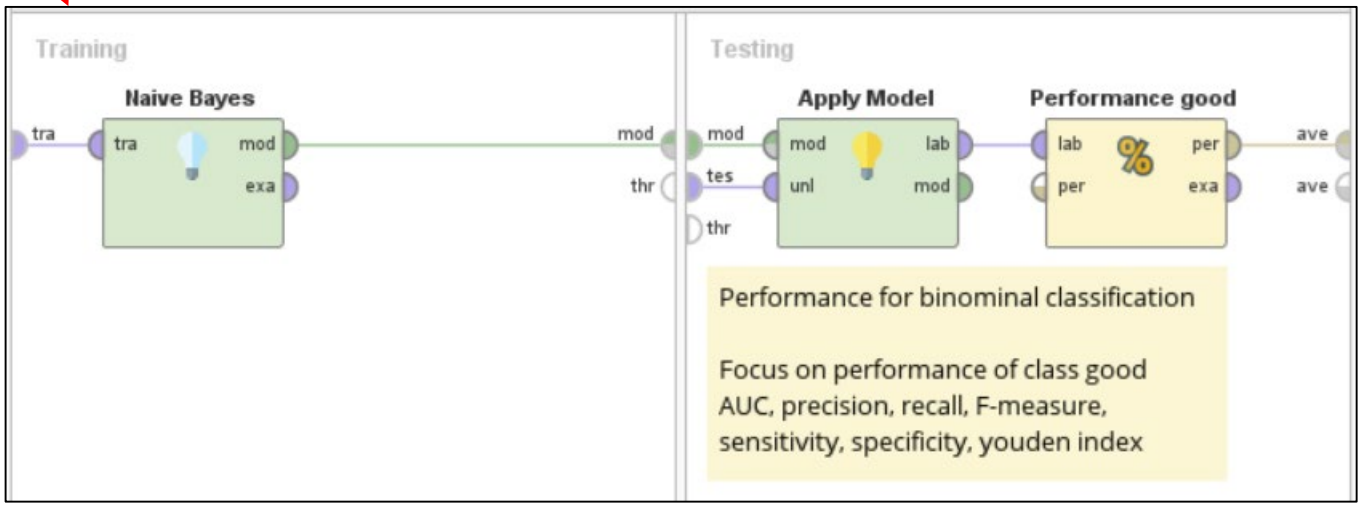
Example (6) : ROC and other binary metrics



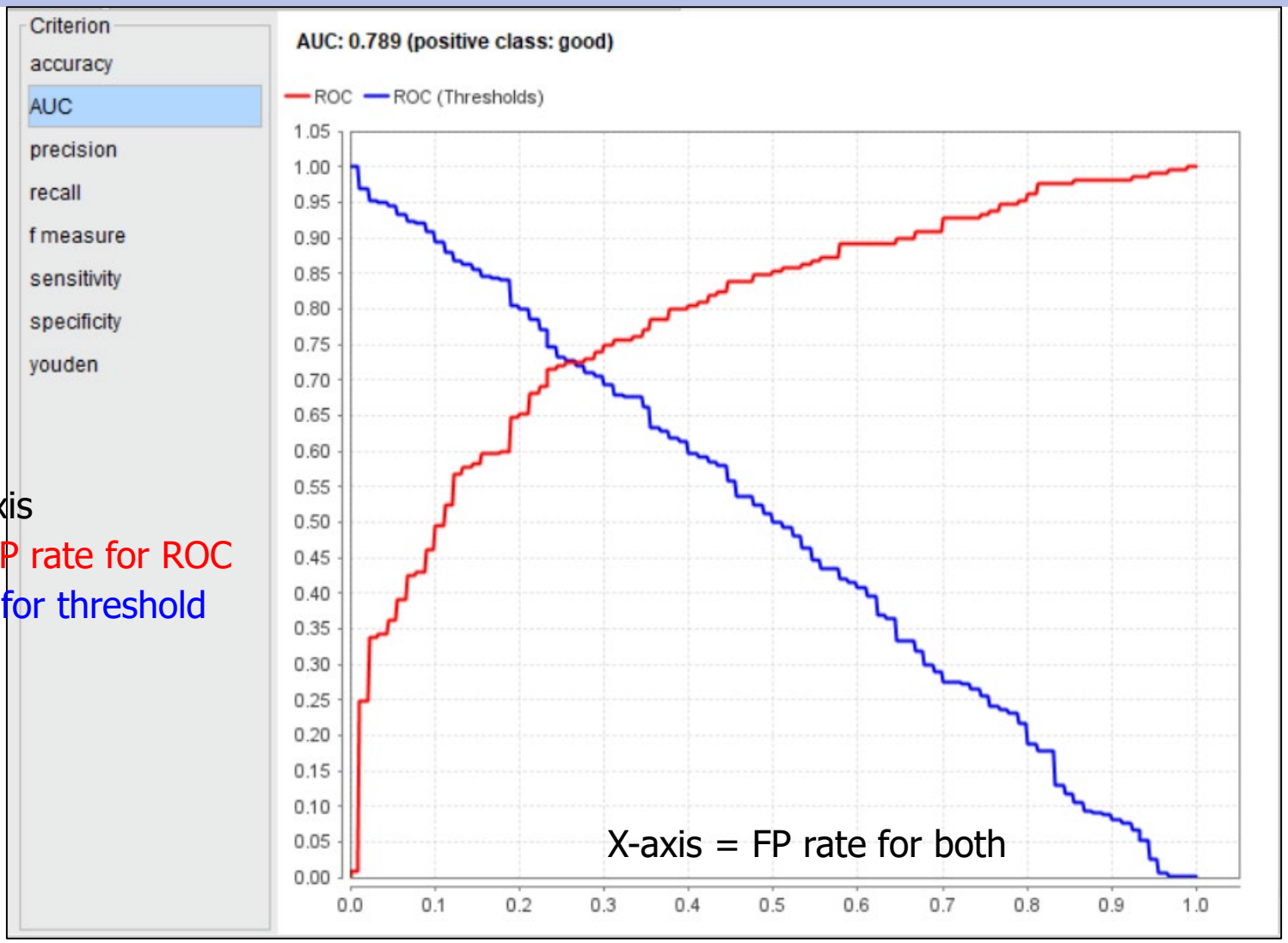
In RapidMiner, ROC and some other performance metrics (especially those based on TP, TN, FP, FN) are available only in binary classification and for positive class


Selection 1
Split Validation good
inp
tra
mod
tra
ave
ave
out
Split Validation bad
inp
inp
tra
mod
tra
ave
ave
out
out
Selection 2
Compare ROCs good
inp
exa
exa
roc
out
Compare ROCs bad
inp
inp
exa
exa
roc
out
out
Set #folds for cross validation
Or #folds = -1 for split validation
Compare ROCs of different classifiers


Compare ROCs good
Naive Bayes (3)
tra
tra
mod
exa
mod
Rule Induction
tra
tra
tra
mod
exa
mod
mod

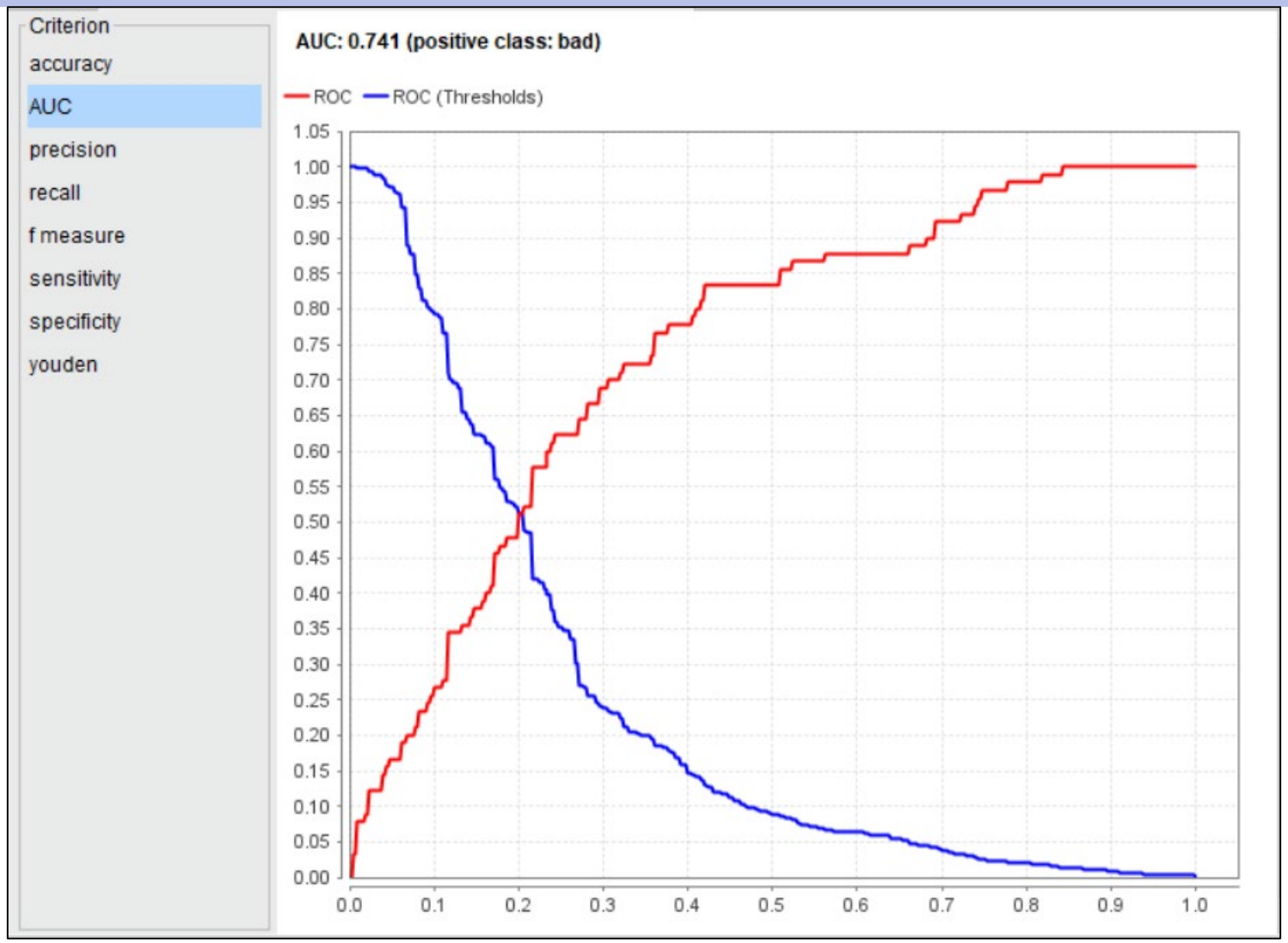

Training
Naive Bayes
tra
tra
mod
exa
mod
thr
Testing
Apply Model
mod
tes
thr
mod
unl
lab
mod
Performance good
lab
per
per
exa
ave
ave
Performance for binominal classification
Focus on performance of class good
AUC, precision, recall, F-measure,
sensitivity, specificity, youden index

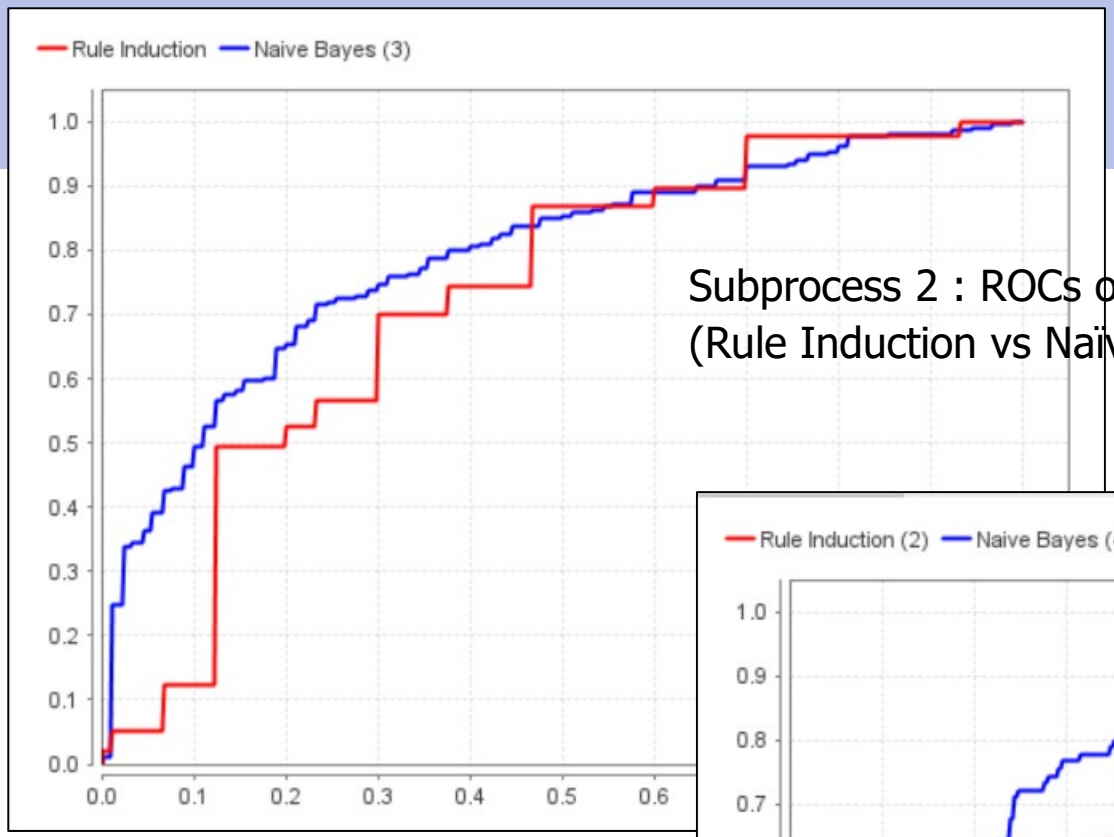
Subprocess 1 : ROC of class good (Naïve Bayes)



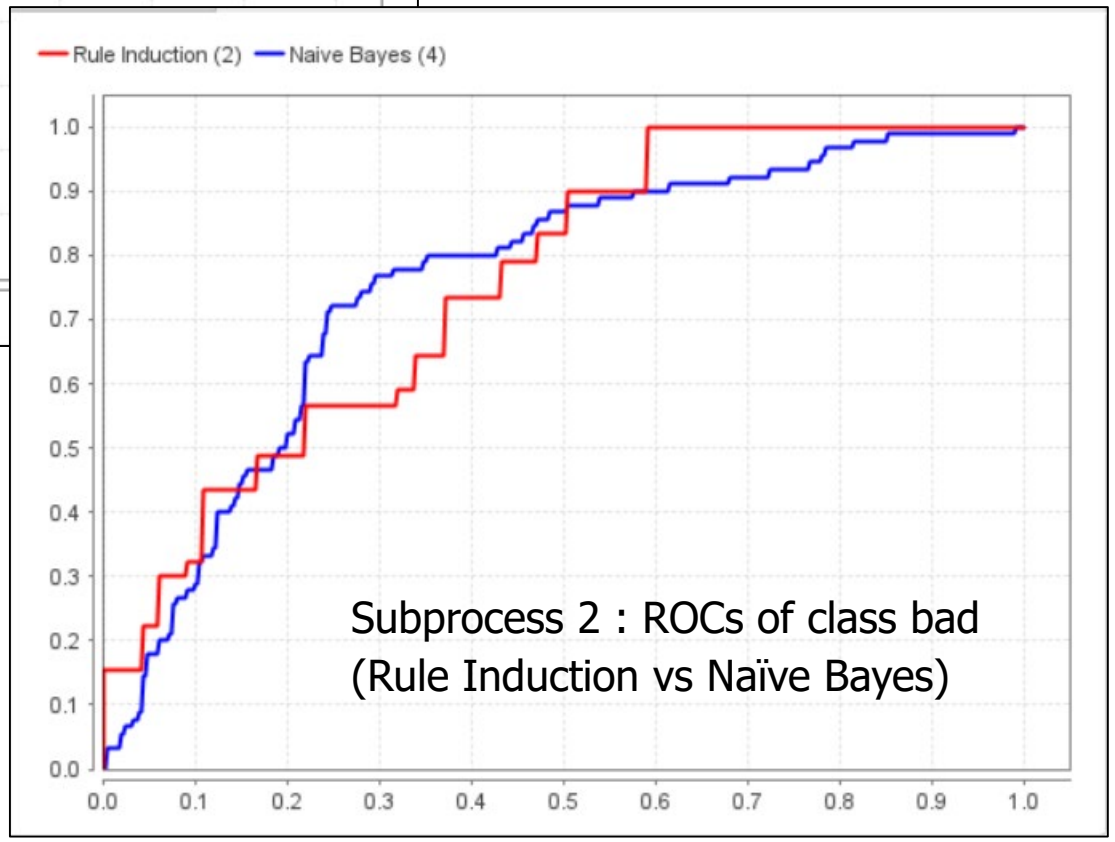
Y-axis
= TP rate for ROC
= t for threshold

Subprocess 1 : ROC of class bad (Naïve Bayes)





Subprocess 2 : ROCs of class good
(Rule Induction vs Naïve Bayes)



Subprocess 2 : ROCs of class bad
(Rule Induction vs Naïve Bayes)